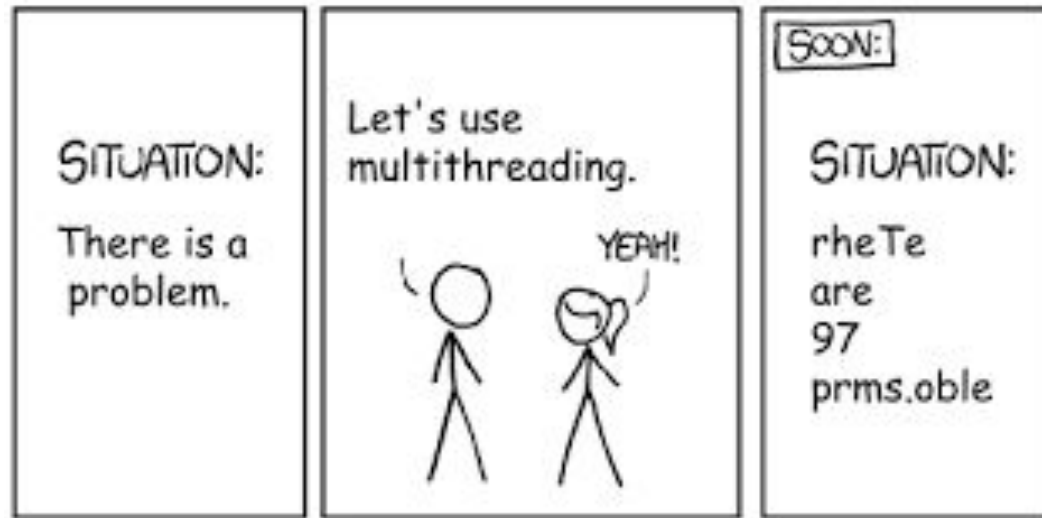


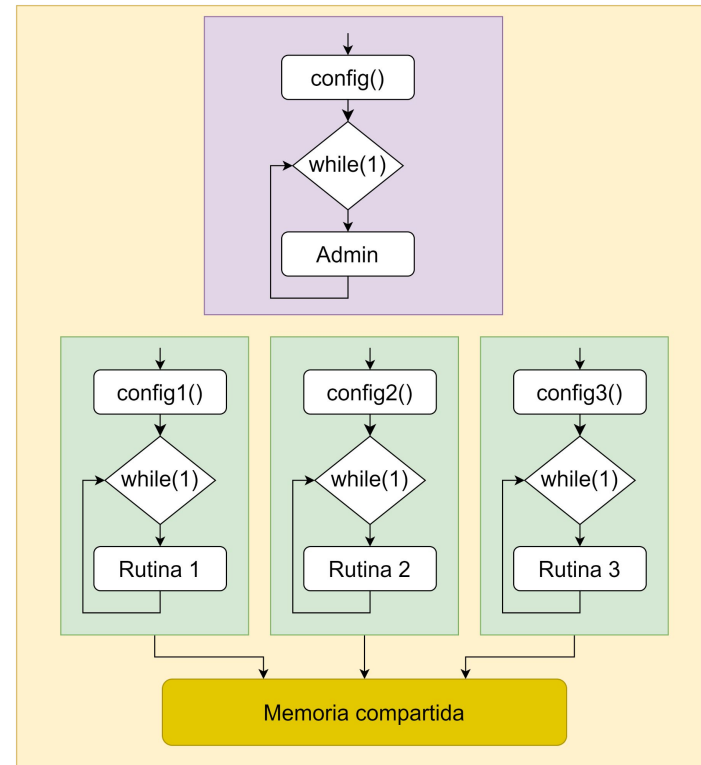
Concurrencia y manejo de recursos compartidos



Concurrencia

A modo de repaso...:

- Es la ejecución de varios (dos o más) flujos de código que se ejecutan en un mismo sistema, c/u realizando acciones específicas, dando la impresión de que se ejecutan todos al mismo tiempo...a estos flujos de código los llamaremos *Tareas*
- Cada tarea se apropia temporalmente del CPU y accede a los recursos del sistema (procesamiento, memoria, periféricos, etc...)
- En algún momento la tarea cede (por un rato) el uso del CPU a otra tarea
- La tarea que cede momentáneamente el CPU a otra (voluntaria o involuntariamente), más tarde debe retomar con la ejecución de la instrucción siguiente a la última que ejecutó, como si nada hubiera sucedido. Para esto se guarda una copia de los registros del CPU (contexto)

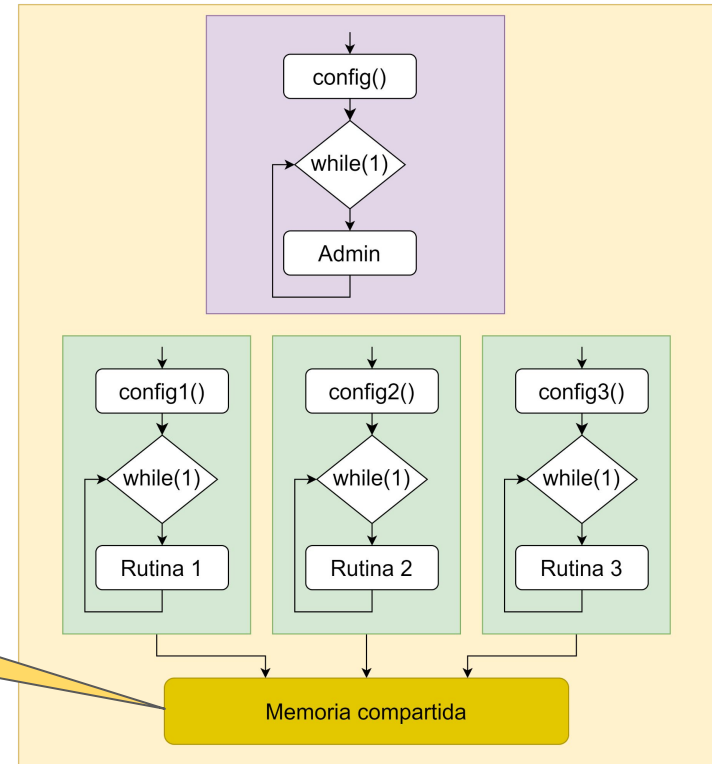


Concurrencia

Este esquema concurrente con tareas que son interrumpidas temporalmente y que luego son reanudadas en la instrucción en que fueron interrumpidas ocurre comúnmente en dos situaciones

- **Background/Foreground (1 o más ISRs)**
- **Multithreading con RTOS (más ISRs)**

Memoria + Periféricos + CPU Compartidos



Problemas comunes en ejecución concurrente

- Condiciones de carrera
 - Acceso al mismo HW
 - Accesos r-m-w no atómicos a variables globales/static
 - Llamadas a funciones no-reentrant / no thread-saves
- Starvation
 - Mal diseño en la asignación de prioridades
 - Inversión de prioridades
- Deadlocks
 - Bloqueos mutuos entre dos o más tareas

Repaso: condiciones de carrera en arquitecturas FG/BG

- Usar **volatile** en datos compartidos entre FG y BG
- Usar **volatile** en datos/direcciones asociadas a I/O
- **Deshabilitar interrupciones temporalmente** al modificar/leer datos compartidos.

`__disable_irq()` / `__enable_irq()` (*cuidado con el anidamiento!!!*)

- Usar **copias** de variables

Repaso: condiciones de carrera en arquitecturas FG/BG

- **Deshabilitar interrupciones temporalmente** al modificar/leer datos compartidos.

`__disable_irq()` / `__enable_irq()` (*cuidado con el anidamiento!!!*)

→ Al código que accede al recurso protegido se lo llama “**sección crítica**”

→ Al mecanismo por el cual solo una tarea puede acceder a la vez a la **sección crítica** se lo denomina **Exclusión mutua**

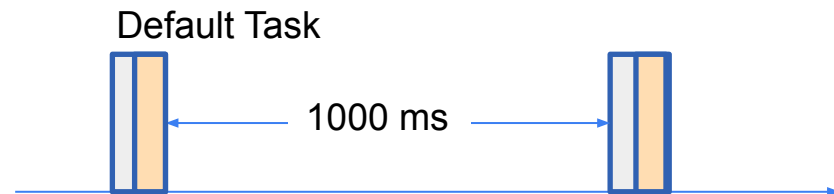
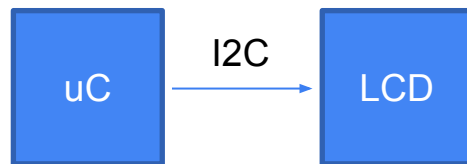
Repaso: condiciones de carrera en arquitecturas FG/BG

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){  
    ...  
    disable_irq();  
    flag = 1;  
    strncpy(str_buffer, "botón presionado\n", 50);  
    __enable_irq();  
}
```

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim){  
    ...  
    disable_irq();  
    flag = 1;  
    strncpy(str_buffer, "timeout\n", 50);  
    __enable_irq();  
}
```

```
volatile char str_buffer[50];  
volatile uint8_t flag=0;  
int main(void){  
    while (1){  
        char local_buffer[50];  
        uint8_t local_flag;  
        __disable_irq();  
        local_flag = flag;  
        if(local_flag) { strncpy(local_buffer, str_buffer, 50); flag=0; }  
        __enable_irq();  
        if(local_flag){ HAL_UART_Transmit(&huart1, local_buffer, strlen(local_buffer), HAL_MAX_DELAY);  
        }  
    }  
}
```

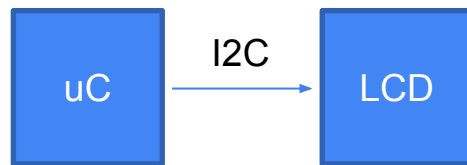
Acceso concurrente al Hardware



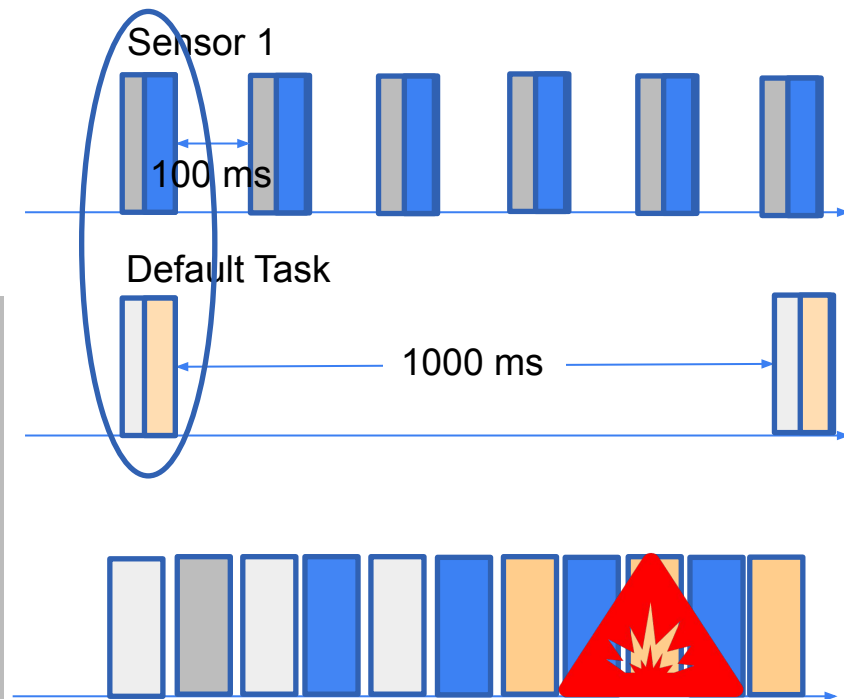
```
void ImprimirLinea(char *s, uint8_t linea) {  
    /* "borra" la línea para poder escribir en limpio */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts("          ", &Font_7x10, 1);  
  
    /* imprime la cadena */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts(s, &Font_7x10, 1);  
  
    /*actualiza pantalla*/  
    SSD1306_UpdateScreen();  
}
```

```
void StartDefaultTask(void *argument) {  
    for (;;) {  
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);  
  
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)) {  
            ImprimirLinea("LED = OFF", 1);  
        } else {  
            ImprimirLinea("LED = ON", 1);  
        }  
  
        osDelay(1000);  
    }  
}
```

Acceso concurrente al Hardware



```
void entrySensor1(void *argument) {  
    int sensor = 0;  
    char val[20] = "sensor1 = ";  
    for (;;) {  
        /* convierte el valor del sensor a ascii y lo  
        concatena al final de "sensor2 = " */  
        itoa(sensor, val + 10, 10);  
  
        ImprimirLinea(val, 2);  
  
        sensor++;  
        osDelay(100);  
    }  
}
```



Problemas

- Acceso c

U



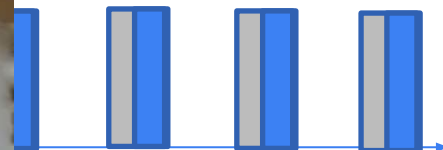
```
void entrySensor1(void
  int sensor = 0;
  char val[20] = "sensor1 = ";
  for (;;) {
    /* convierte el valor del sensor a string
    concatena al final de la cadena
    itoa(sensor, val, 10);
```

```
ImprimirLinea(val);
```

```
  sensor++;
  osDelay(100);
```

```
}
```

```
}
```



Identificación del recurso compartido

Una “solución” al problema de los **múltiples accesos en simultáneo al mismo recurso** es detectar la **región de código crítico** e implementar un **bloqueo o exclusión**, que garantice **que sólo se accede de a una tarea por vez al recurso** → **Exclusión Mutua**

```
void ImprimirLinea(char *s, uint8_t linea) {  
  
    /* "borra" la línea para poder escribir en limpio */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts("                ", &Font_7x10, 1);  
  
    /* imprime la cadena */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts(s, &Font_7x10, 1);  
  
    /*actualiza pantalla*/  
    SSD1306_UpdateScreen();  
  
}
```

Identificación del recurso compartido

Una “solución” al problema de los **múltiples accesos en simultáneo al mismo recurso** es detectar la **región de código crítico** e implementar un **bloqueo o exclusión**, que garantice **que sólo se accede de a una tarea por vez al recurso** → **Exclusión Mutua**

```
void ImprimirLinea(char *s, uint8_t linea) {  
  
    /* "borra" la línea para poder escribir en limpio */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts("                ", &Font_7x10, 1);  
  
    /* imprime la cadena */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts(s, &Font_7x10, 1);  
  
    /*actualiza pantalla*/  
    SSD1306_UpdateScreen();  
  
}
```

Identificación del recurso compartido

Una “solución” al problema de los **múltiples accesos en simultáneo al mismo recurso** es detectar la **región de código crítico** e implementar un **bloqueo o exclusión**, que garantice **que sólo se accede de a una tarea por vez al recurso**

```
void ImprimirLinea(char *s, uint8_t line)
{
    /* "borra" la línea pasada */
    SSD1306_GotoXY(0, line - 1);
    SSD1306_Puts(" ");

    /* imprime la cadena */
    SSD1306_GotoXY(0, line + 14 * (line - 1));
    SSD1306_Puts(s, 0);

    /*actualiza pantalla*/
    SSD1306_UpdateScreen();
}
```

Todas las funciones de la librería SSD1306 tienen el potencial de acceder a memoria global y/o al hardware mediante I2C:

No son reentrantes/thread-safe

Implementación de la Exclusión Mutua en RTOS

- **Suspensión de interrupciones general**
- **Suspensión de interrupciones al nivel del RTOS**
- **Suspensión del scheduler**
- **Semáforo binario / Mutex**
- **No compartir recursos**

Exclusión mutua con **Suspensión de interrupciones**

Cuando se identifica una sección crítica, esta puede protegerse de cambios de contexto indeseados deshabilitando interrupciones (los cambios de contexto solo pueden realizarse si ocurre una interrupción)

La sección crítica encerrada por estas debe ser breve y el tiempo de ejecución lo más determinístico posible, ya que se incrementa la **latencia** de todas las interrupciones.

Exclusión mutua con **Suspensión de interrupciones general**

Si se corre el riesgo de que al menos una ISR acceda al recurso compartido deberá aplicarse la Exclusión Mutua mediante la misma solución que en un esquema FG/BG sin RTOS, usando `__disable_irq()` / `__enable_irq()` que deshabilita todas las fuentes de interrupción (enmascarables) y **no tienen en cuenta el efecto de anidamiento**.

```
void ImprimirLinea(char *s, uint8_t linea) {  
    __disable_irq();  
    /* "borra" la línea para poder escribir en limpio */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts("                ", &Font_7x10, 1);  
  
    /* imprime la cadena */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts(s, &Font_7x10, 1);  
  
    /*actualiza pantalla*/  
    SSD1306_UpdateScreen();  
    __enable_irq();  
}
```

Exclusión mutua con **Suspensión de interrupciones al nivel del RTOS**

Si el recurso compartido es accedido únicamente desde las tareas (background) conviene utilizar el mecanismo de FreeRTOS para deshabilitar y rehabilitar interrupciones: `taskENTER_CRITICAL()` y `taskEXIT_CRITICAL()` que deshabilita únicamente las fuentes de interrupción con prioridad compatible con el RTOS (≥ 5)

```
void ImprimirLinea(char *s, uint8_t linea) {  
    taskENTER_CRITICAL();  
    /* "borra" la línea para poder escribir en limpio */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts("                ", &Font_7x10, 1);  
  
    /* imprime la cadena */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts(s, &Font_7x10, 1);  
  
    /*actualiza pantalla*/  
    SSD1306_UpdateScreen();  
    taskEXIT_CRITICAL();  
}
```

Exclusión mutua con **Suspensión de interrupciones al nivel del RTOS**

Si el recurso compartido es accedido únicamente desde las tareas (background) conviene utilizar el mecanismo de FreeRTOS para deshabilitar y rehabilitar interrupciones. Se utiliza la función `taskEXIT_CRITICAL()` que deshabilita únicamente las fuentes de interrupción compatibles con el RTOS (≥ 5)

```
void ImprimirLinea(char *s, uint8_t linea)
{
    taskENTER_CRITICAL();

    /* "borra" la línea para SSD1306 */
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));
    SSD1306_Puts(" ");

    /* imprime la cadena */
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));
    SSD1306_Puts(s, &Font_7x10, 1);

    /*actualiza pantalla*/
    SSD1306_UpdateScreen();

    taskEXIT_CRITICAL();
}
```

Enmascara **atómicamente** todas las fuentes de interrupción con prioridad menor a `configMAX_SYSCALL_INTERRUPT_PRIORITY` (mayor valor numérico). Solo la tarea que logre ejecutarla tendrá acceso al código crítico.

QUEDA SUSPENDIDA LA EJECUCIÓN DEL RTOS (y así los cambios de contexto)

Exclusión mutua con **Suspensión de interrupciones al nivel del RTOS**

ANIDAMIENTO: En FreeRTOS es seguro anidar secciones críticas, ya que se lleva un contador de llamadas y solo se rehabilitan las interrupciones cuando el contador vuelve a cero. Sin este contador, el primer llamado a `taskEXIT_CRITICAL()` habilitaría las interrupciones antes de lo deseado.

```
void ImprimirLinea(char *s, uint8_t linea) {  
    taskENTER_CRITICAL();  
    /* "borra" la línea para poder escribir en ella */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts(" ", &Font_7x10, 1);  
  
    /* imprime la cadena */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts(s, &Font_7x10, 1);  
  
    /*actualiza pantalla*/  
    SSD1306_UpdateScreen();  
    taskEXIT_CRITICAL();  
}
```

portDISABLE_INTERRUPTS();
uxCriticalNesting++;

uxCriticalNesting--;
if(uxCriticalNesting == 0)
 {portENABLE_INTERRUPTS();}

Exclusión mutua con **Suspensión de interrupciones al nivel del RTOS**

LATENCIA: Como el tiempo de ejecución de una sección crítica debe ser corto, la solución propuesta no es la más adecuada, ya que la comunicación con la pantalla LCD demora decenas de ms. Y durante ese tiempo el sistema está bloqueado en una única tarea

```
void ImprimirLinea(char *s, uint8_t linea) {  
    taskENTER_CRITICAL();  
    /* "borra" la línea para poder escribir en limpio */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts("                ", &Font_7x10, 1);  
  
    /* imprime la cadena */  
    SSD1306_GotoXY(0, 0 + 14 * (linea));  
    SSD1306_Puts(s, &Font_7x10, 1);  
  
    /*actualiza pantalla*/  
    SSD1306_UpdateScreen();  
    taskEXIT_CRITICAL();  
}
```

Comunicación I2C con el dispositivo que demora decenas de milisegundos.

Exclusión mutua con **Suspensión del Scheduler**

Otra forma de proteger secciones críticas es suspendiendo el scheduler (esto previene cambios de contexto entre tareas pero no entre tarea/ISR).

Si el recurso compartido no es accedido desde ninguna ISR conviene esta alternativa.

En lugar de utilizar `taskENTER_CRITICAL()` / `taskEXIT_CRITICAL()` Debe usarse alguno de los siguientes pares de funciones:

- API FreeRTOS: `vTaskSuspendAll()` / `xTaskResumeAll()`
- API CMSIS-RTOS V2: `osKernelLock()` / `osKernelRestoreLock()`

Exclusión mutua **con Semáforos Binarios**

Los Semáforos son mecanismos de sincronización que ofrecen acceso atómico a un contador que se decrementa típicamente al **tomar** el semáforo (sino bloquea hasta lograrlo) y se incrementa al **entregar** el semáforo.

Los semáforos binarios sólo admiten dos valores (0: recurso tomado, 1: recurso libre), por lo que **solo una tarea o ISR podrá acceder al recurso** y/o ejecutar el código crítico correspondiente, mientras que el resto de las tareas pasará a un estado de bloqueo cuando intente tomar el semáforo.

Es un mecanismo que **solo involucra a las tareas que desean hacer uso del recurso**, mientras que las otras se ejecutan libremente.

PUEDE USARSE CON MUCHA PRECAUCIÓN DESDE ISRs

Exclusión mutua **con Semáforos Binarios**

```
void ImprimirLinea(char *s, uint8_t linea) {  
    /*Si el semáforo ya fue tomado, La tarea se bloqueará a partir de la próxima línea, hasta que  
    otra tarea lo libere*/  
    osSemaphoreAcquire(sbHandle, osWaitForever);    //xSemaphoreTake( sbHandle, portMAX_DELAY );  
  
    /* "borra" la línea para poder escribir en limpio */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts("                ", &Font_7x10, 1);  
  
    /* imprime la cadena */  
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
    SSD1306_Puts(s, &Font_7x10, 1);  
  
    /*actualiza pantalla*/  
    SSD1306_UpdateScreen();  
  
    /* libera el semáforo para desbloquear otra tarea que esté bloqueada esperando el acceso al  
    display */  
    osSemaphoreRelease( sbHandle );    //xSemaphoreGive( sbHandle );  
}
```

Exclusión mutua **con Mutex**

Funciona similar a un **semáforo binario**. Pero es un objeto provisto por el SO que permite implementar específicamente la **exclusión mutua sobre un recurso, basada en la posesión de un token**.

Solo una tarea podrá poseer el token, acceder al recurso y ejecutar el código crítico correspondiente, mientras que el resto de las tareas pasará a un estado de bloqueo cuando intente tomar el token.

Cuando la tarea que posee el token lo libera, solo una de las que están en espera lo toma y es la única que puede acceder al recurso.

Es un mecanismo que solo involucra a las tareas que desean hacer uso del recurso, mientras que las otras se ejecutan libremente. Las tareas bloqueadas a la espera del mutex no consumen ciclos de CPU.

NO PUEDE USARSE DESDE ISRs

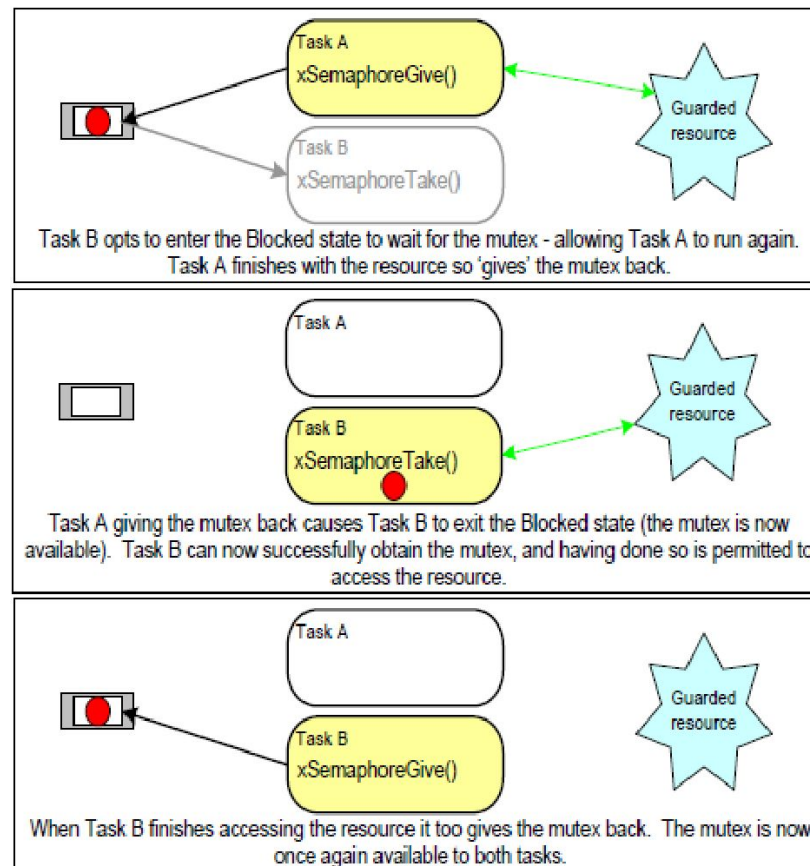
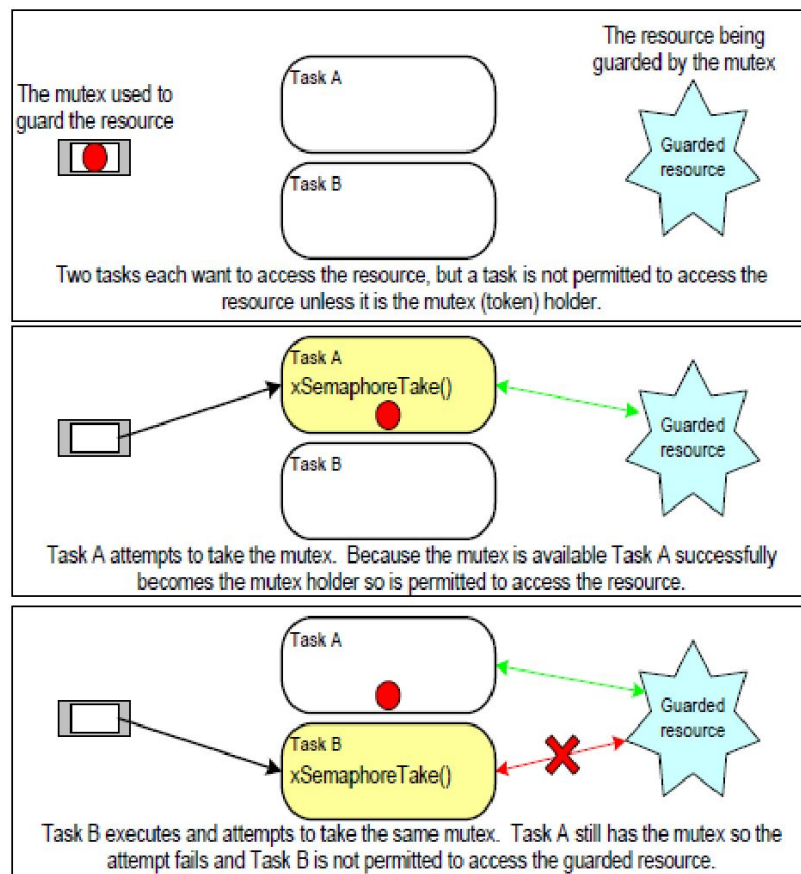


Figure 63. Mutual exclusion implemented using a mutex

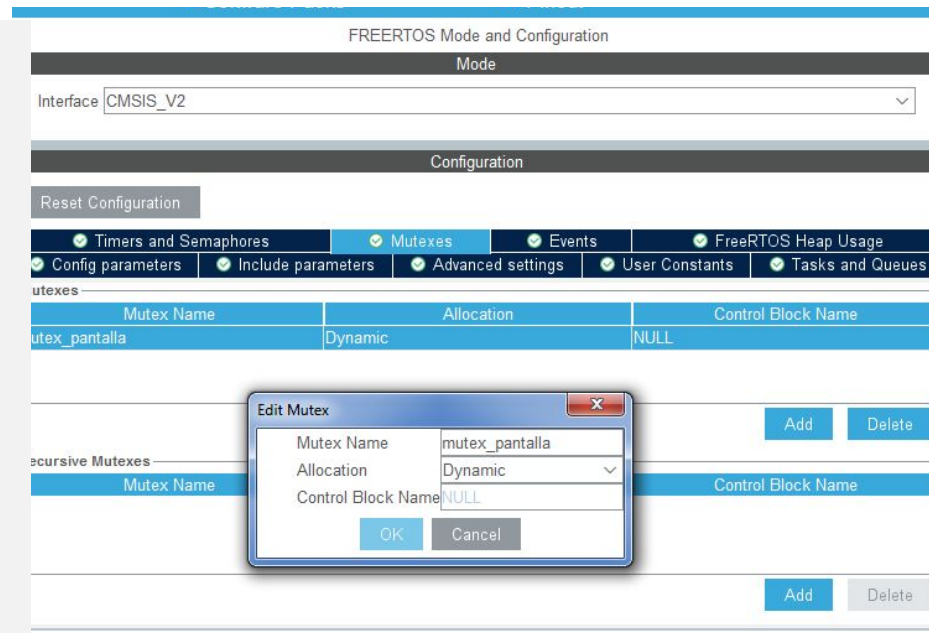
Exclusión mutua **con Mutex**

	API FreeRTOS	API CMSIS-RTOS V2
tipo	SemaphoreHandle_t	osMutexId_t
creación	xSemaphoreCreateMutex();	osMutexNew (*attr)
posesión	xSemaphoreTake (xSemaphore, xBlockTime)	osMutexAcquire (mutex_id, timeout)
devolución	xSemaphoreGive (xMutex)	osMutexRelease (mutex_id)

Para poder utilizarlo deberá configurarse
configUSE_MUTEXES en 1

Exclusión mutua **con Mutex**

```
void ImprimirLinea(char *s, uint8_t linea) {  
  
    if(osOK==osMutexAcquire(mutex_pantallaHandle, 1000))  
    {  
        /* "borra" la línea para poder escribir en limpio */  
        SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
        SSD1306_Puts("                ", &Font_7x10, 1);  
  
        /* imprime la cadena */  
        SSD1306_GotoXY(0, 0 + 14 * (linea - 1));  
        SSD1306_Puts(s, &Font_7x10, 1);  
  
        /*actualiza pantalla*/  
        SSD1306_UpdateScreen();  
        osMutexRelease(mutex_pantallaHandle);  
    }  
}
```

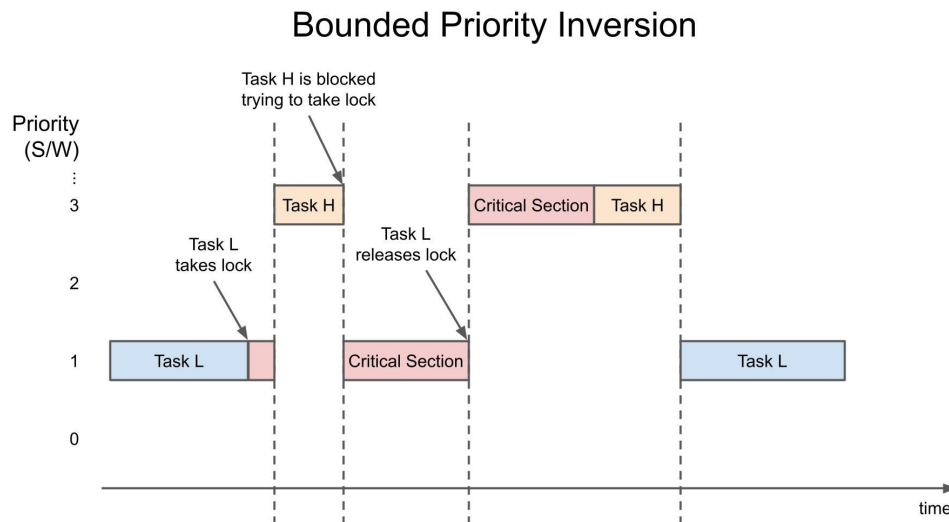


Exclusión mutua **con Mutex** – problemas asociados

- Inversión de prioridades
- Deadlocks
- Mutex anidados (recursivos)

Exclusión mutua **con Mutex** – problemas asociados

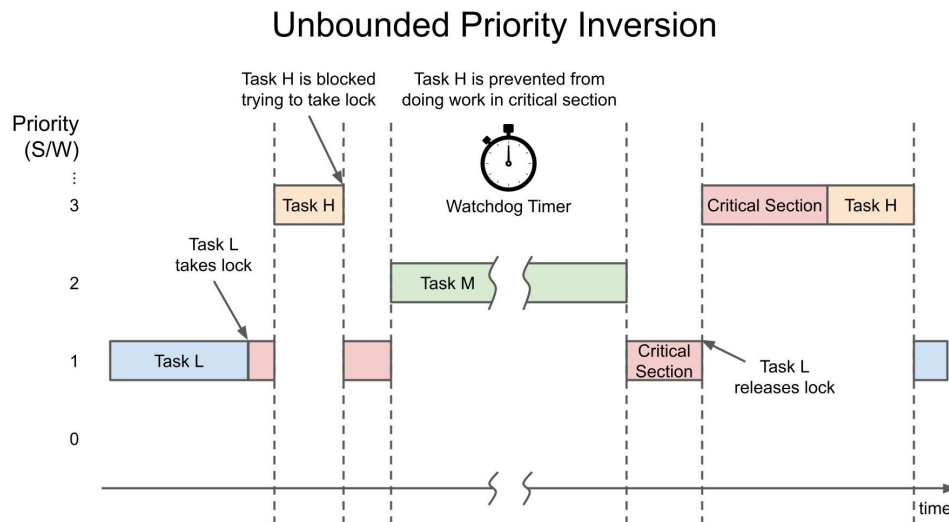
- Inversión de prioridades



Este es el comportamiento deseado, con un bloqueo acotado por la duración de la sección crítica (que debe hacerse lo más corta posible)

Exclusión mutua **con Mutex** – problemas asociados

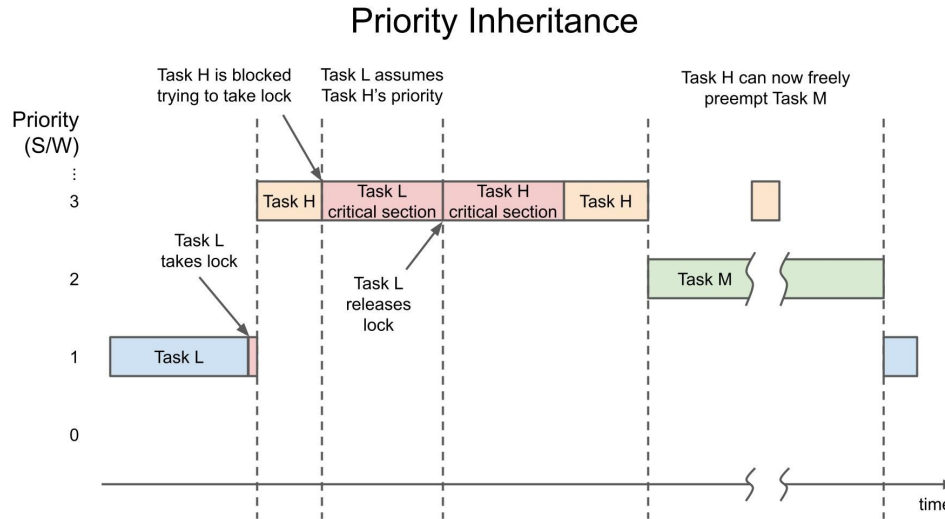
- Inversión de prioridades



Si hay otras tareas, el bloqueo deja de estar acotado, se pierde determinismo y se invierten las prioridades

Exclusión mutua **con Mutex** – problemas asociados

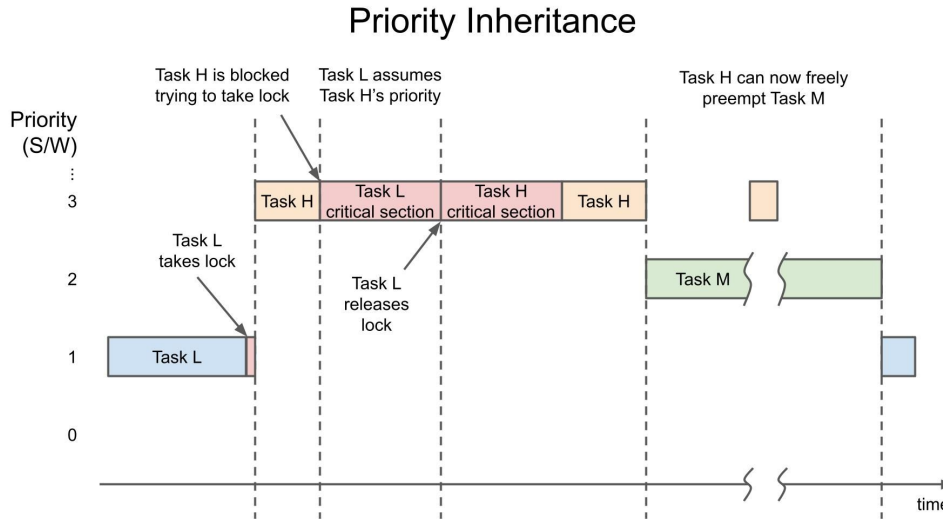
- Inversión de prioridades



La **herencia de prioridades** soluciona el bloqueo no acotado, promoviendo la tarea propietaria del mutex a la prioridad más alta de las tareas que se encuentran bloqueadas esperando el acceso al recurso

Exclusión mutua **con Mutex** – problemas asociados

- Inversión de prioridades



Esta es una diferencia importante entre mutex y semáforo binario; explica porqué solo puede usarse con tareas y no desde ISRs

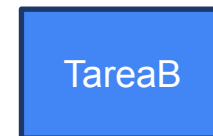
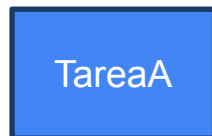
La **herencia de prioridades** soluciona el bloqueo no acotado, promoviendo la tarea propietaria del mutex a la prioridad más alta de las tareas que se encuentran bloqueadas esperando el acceso al recurso

Exclusión mutua **con Mutex** – problemas asociados

- **Deadlock**

```
main_TareaA(){  
    xSemaphoreTake(Recurso1);  
    //hace algo con Recurso1  
    xSemaphoreTake(Recurso2);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

```
main_TareaB(){  
    xSemaphoreTake(Recurso2);  
    //hace algo con Recurso2  
    xSemaphoreTake(Recurso1);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

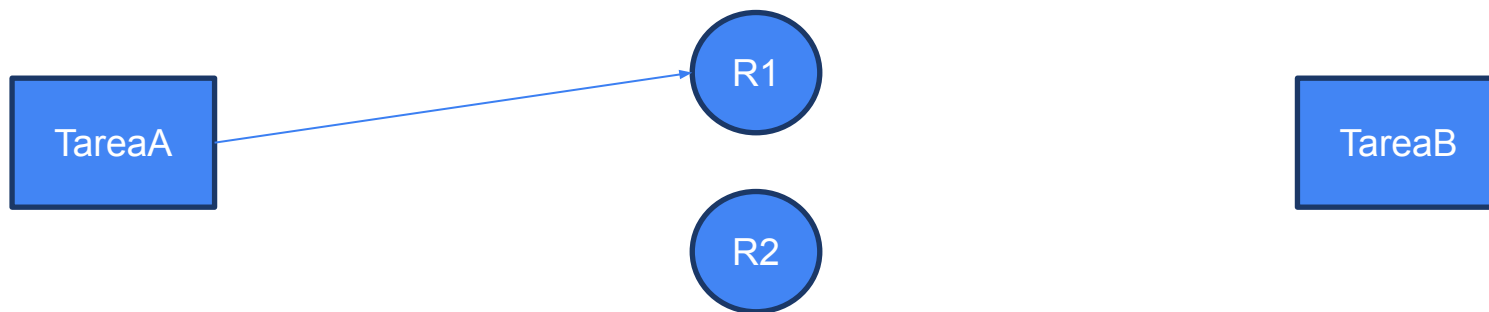


Exclusión mutua **con Mutex** – problemas asociados

- **Deadlock**

→ `main_TareaA(){
 xSemaphoreTake(Recurso1);
 //hace algo con Recurso1
 xSemaphoreTake(Recurso2);
 //hace algo con ambos Recursos
 xSemaphoreGive(Recurso1);
 xSemaphoreGive(Recurso2);
}`

`main_TareaB(){
 xSemaphoreTake(Recurso2);
 //hace algo con Recurso2
 xSemaphoreTake(Recurso1);
 //hace algo con ambos Recursos
 xSemaphoreGive(Recurso1);
 xSemaphoreGive(Recurso2);
}`

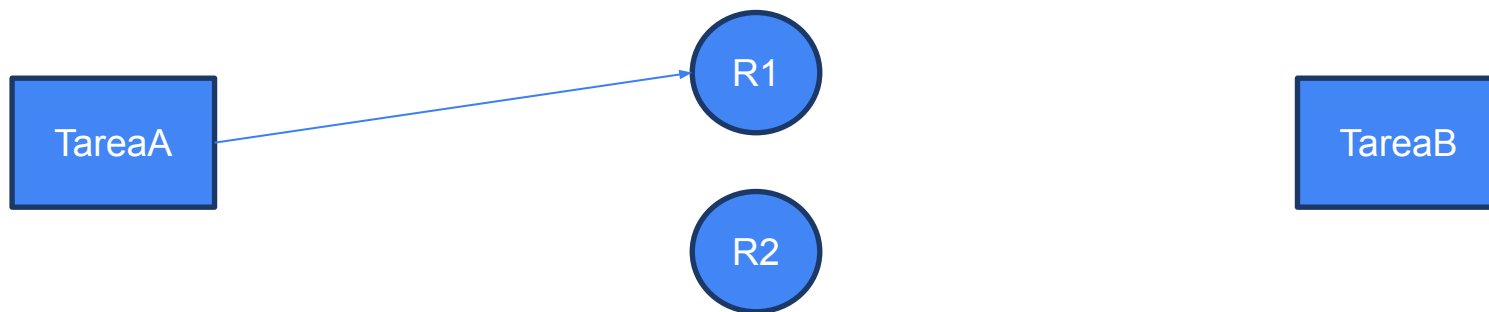


Exclusión mutua **con Mutex** – problemas asociados

- **Deadlock**

```
main_TareaA(){  
    xSemaphoreTake(Recurso1);  
    //hace algo con Recurso1  
    xSemaphoreTake(Recurso2);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

```
main_TareaB(){  
    xSemaphoreTake(Recurso2);  
    //hace algo con Recurso2  
    xSemaphoreTake(Recurso1);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

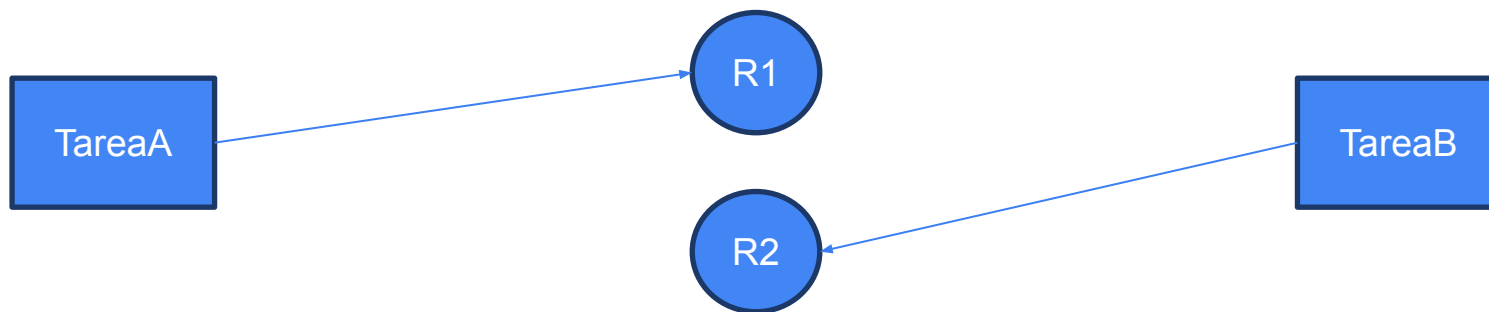


Exclusión mutua **con Mutex** – problemas asociados

- **Deadlock**

```
main_TareaA(){  
    xSemaphoreTake(Recurso1);  
    //hace algo con Recurso1  
    xSemaphoreTake(Recurso2);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

```
main_TareaB(){  
    xSemaphoreTake(Recurso2);  
    //hace algo con Recurso2  
    xSemaphoreTake(Recurso1);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

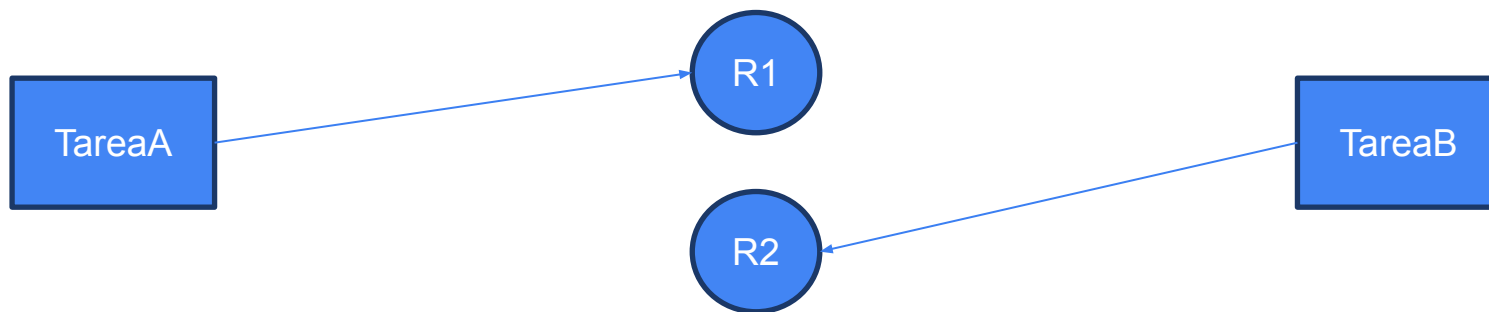


Exclusión mutua **con Mutex** – problemas asociados

- **Deadlock**

```
main_TareaA(){  
    xSemaphoreTake(Recurso1);  
    //hace algo con Recurso1  
    xSemaphoreTake(Recurso2);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

```
main_TareaB(){  
    xSemaphoreTake(Recurso2);  
    //hace algo con Recurso2  
    xSemaphoreTake(Recurso1);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

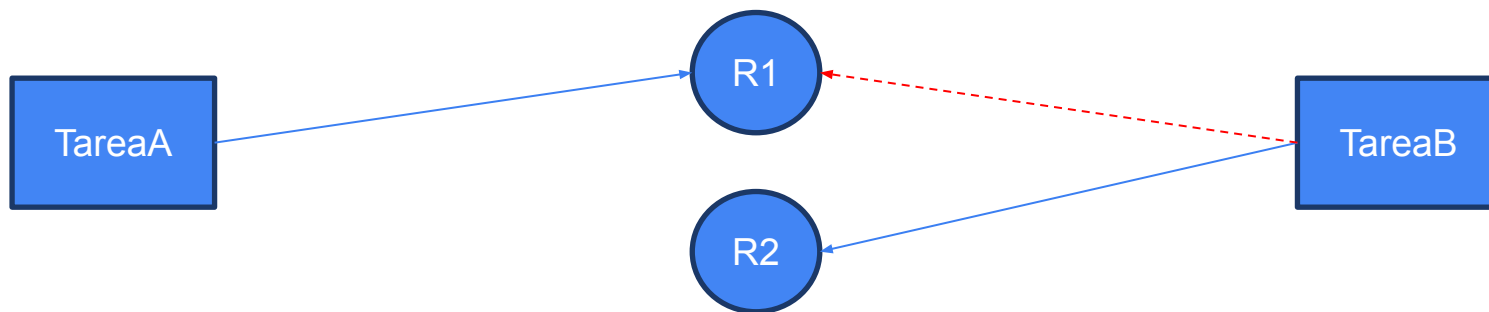


Exclusión mutua **con Mutex** – problemas asociados

- **Deadlock**

```
main_TareaA(){  
    xSemaphoreTake(Recurso1);  
    //hace algo con Recurso1  
    xSemaphoreTake(Recurso2);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

```
main_TareaB(){  
    xSemaphoreTake(Recurso2);  
    //hace algo con Recurso2  
    xSemaphoreTake(Recurso1);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

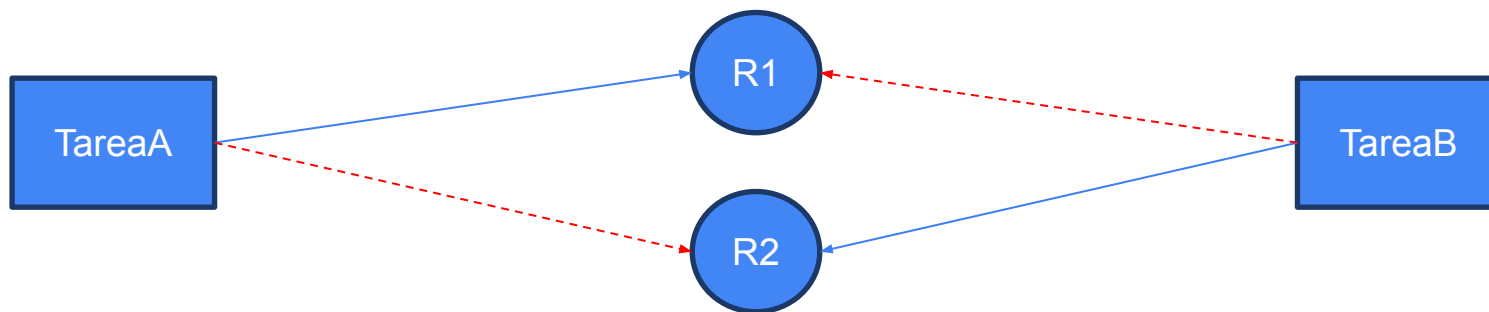


Exclusión mutua **con Mutex** – problemas asociados

- **Deadlock**

```
main_TareaA(){  
    xSemaphoreTake(Recurso1);  
    //hace algo con Recurso1  
    → xSemaphoreTake(Recurso2);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```

```
main_TareaB(){  
    xSemaphoreTake(Recurso2);  
    //hace algo con Recurso2  
    → xSemaphoreTake(Recurso1);  
    //hace algo con ambos Recursos  
    xSemaphoreGive(Recurso1);  
    xSemaphoreGive(Recurso2);  
}
```



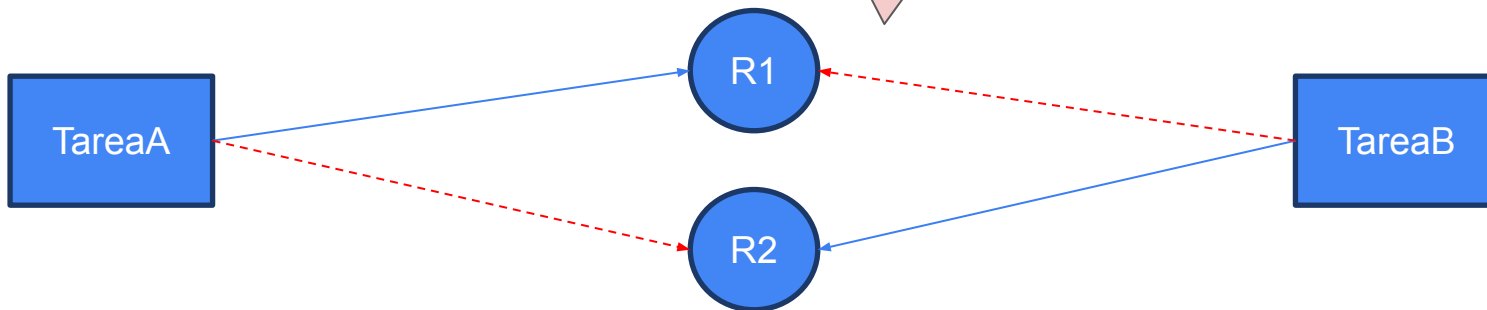
Exclusión mutua **con Mu**

- **Deadlock**

```
main_TareaA  
xSemaph  
//hace  
xSemaph  
//hace algo con amb  
xSemaphoreGive(Recur  
xSemaphoreGive(Rekurs  
}
```



```
//hace algo con ambos Recursos  
xSemaphoreGive(Recurso1);  
xSemaphoreGive(Recurso2);  
}
```



En este punto, ambas tareas quedan bloqueadas.

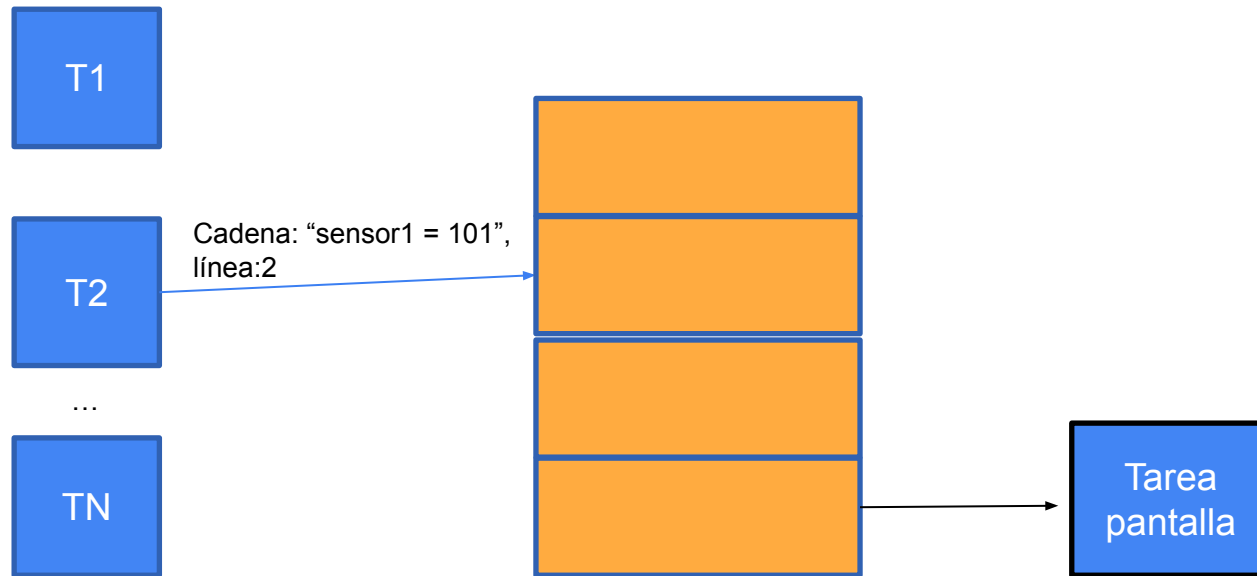
Esto puede suceder tanto con Mutex como con SB.

Puede evitarse con buenas prácticas de diseño:

- Usar timeouts finitos cuando se desea tomar el mutex
- Tomar los mutex en el mismo orden en ambas tareas

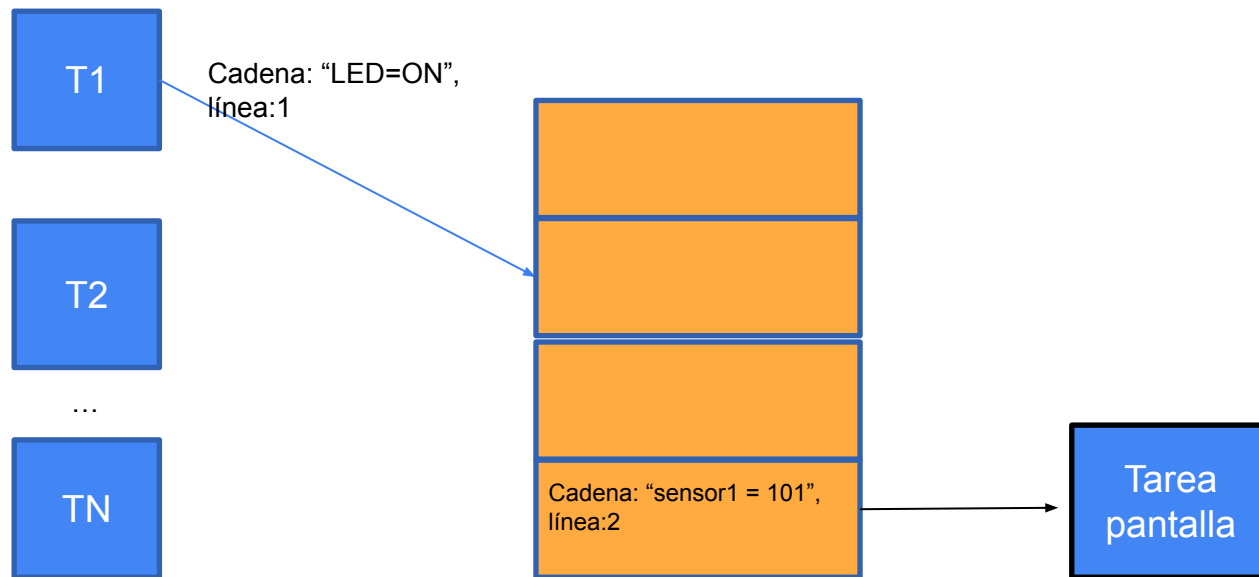
Exclusión mutua extrema ☐ **No compartir recursos**

Una única **tarea de servicio** gestiona el acceso al recurso desde diferentes tareas a través de colas o buffers.



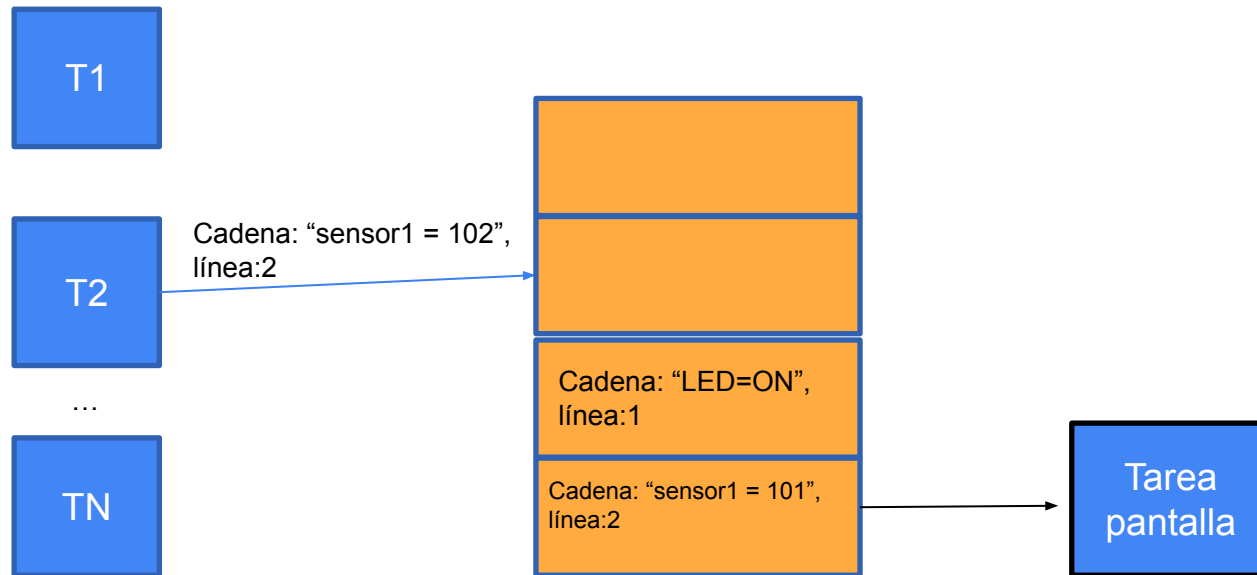
Exclusión mutua extrema ☐ **No compartir recursos**

Una única **tarea de servicio** gestiona el acceso al recurso desde diferentes tareas a través de colas o buffers.



Exclusión mutua extrema ☐ **No compartir recursos**

Una única **tarea de servicio** gestiona el acceso al recurso desde diferentes tareas a través de colas o buffers.



Queue (cola/FIFO)

Es un elemento provisto por el SO que permite implementar y manipular de manera sencilla una cola de mensajes, datos u objetos entre tareas. Posee un tamaño fijo (cantidad de elementos), para almacenar datos de un tipo determinado (tamaño de cada elemento).

Suele utilizarse de manera que solo una tarea consume los mensajes, mientras que una o más tareas escriben en la misma.

Una tarea que desea escribir en un FIFO lleno se bloquea sin consumir ciclos de CPU, y el RTOS la desbloquea cuando se hace lugar en el FIFO (o expira un timeout).

Una tarea que desea leer de una cola vacía se bloquea sin consumir ciclos de CPU y el RTOS la desbloquea cuando se escribe algo nuevo en la cola (o expira un timeout).

El RTOS resuelve la exclusión mutua en caso de que muchas tareas estén queriendo acceder “en simultáneo” para leer/escribir en la cola.

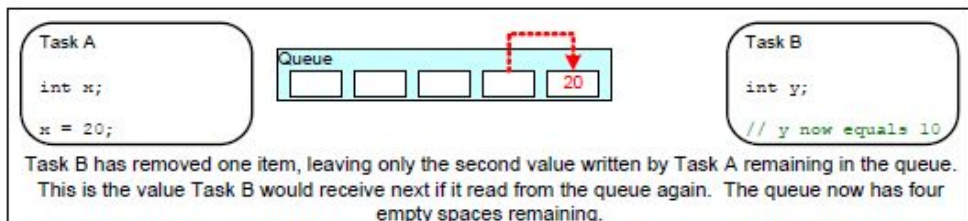
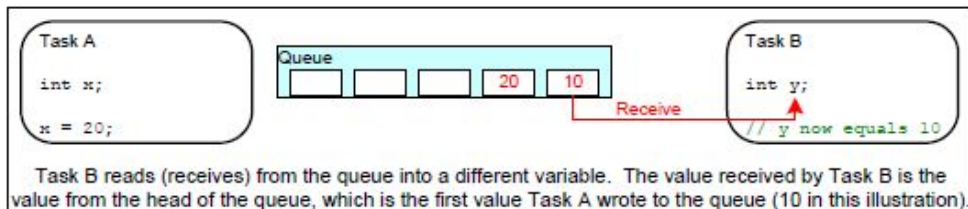
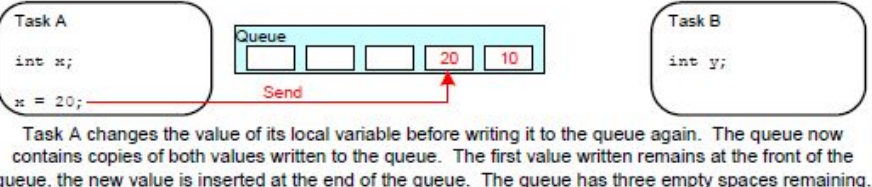
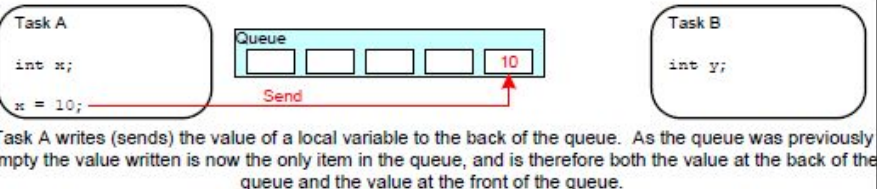
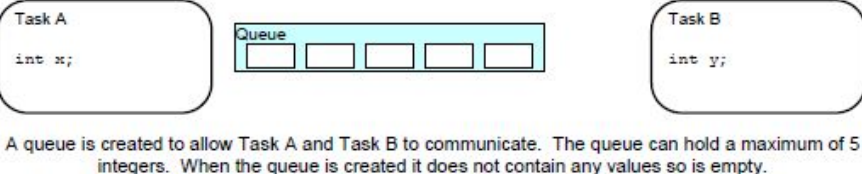


Figure 31. An example sequence of writes to, and reads from a queue

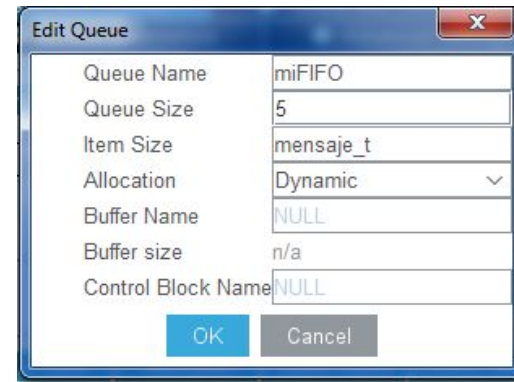
	API FreeRTOS	API CMSIS-RTOS V2
tipo	QueueHandle_t	osMessageQueueId_t
creación	xQueueCreate (msg_count, msg_size)	osMessageQueueNew (msg_count, msg_size, *attr)
escritura	BaseType_t xQueueSend (QueueHandle_t xQueue, const void * pvltemToQueue, TickType_t xTicksToWait); BaseType_t xQueueSendToFront (QueueHandle_t xQueue, const void * pvltemToQueue, TickType_t xTicksToWait);	osStatus_t osMessageQueuePut (osMessageQueueId_t mq_id, const void *msg_ptr, uint8_t msg_prio, uint32_t timeout);
lectura	BaseType_t xQueueReceive (QueueHandle_t xQueue, void *pvBuffer, TickType_t xTicksToWait);	osStatus_t osMessageQueueGet (osMessageQueueId_t mq_id, void *msg_ptr, uint8_t *msg_prio, uint32_t timeout)

	API FreeRTOS	API CMSIS-RTOS V2
tipo	QueueHandle_t	osMessageQueueId_t
creación	xQueueCreate (msg_count, msg_size)	osMessageQueueNew (msg_count, msg_size, *attr)
escritura	BaseType_t xQueueSend (QueueHandle_t xQueue, const void * pvltemToQueue, TickType_t xTicksToWait);  BaseType_t xQueueSendToFront (QueueHandle_t xQueue, const void * pvltemToQueue, TickType_t xTicksToWait); 	osStatus_t osMessageQueuePut (osMessageQueueId_t mq_id, const void *msg_ptr, uint8_t msg_prio, uint32_t timeout); 
lectura	BaseType_t xQueueReceive (QueueHandle_t xQueue, void *pvBuffer, TickType_t xTicksToWait);	osStatus_t osMessageQueueGet (osMessageQueueId_t mq_id, void *msg_ptr, uint8_t *msg_prio, uint32_t timeout)

Exclusión mutua extrema ☐ No compartir recursos

```
typedef struct {char cad[20]; int linea;} mensaje_t;
```

La tarea consumidora de los datos del FIFO será la única que pueda escribir en la pantalla por lo que se puede usar nuevamente la versión insegura de ImprimirLinea()



```
void entryPantalla(void *argument)
{
    mensaje_t dato;

    for(;;)
    {
        osMessageQueueGet( miFIFOHandle, &dato, NULL, osWaitForever);
        ImprimirLinea(dato.cad, dato.linea);
    }
}
```

```
void ImprimirLinea(char *s, uint8_t linea) {
    /* "borra" la línea para poder escribir en limpio */
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));
    SSD1306_Puts(" ", &Font_7x10, 1);
    /* imprime la cadena */
    SSD1306_GotoXY(0, 0 + 14 * (linea - 1));
    SSD1306_Puts(s, &Font_7x10, 1);
    /*actualiza pantalla*/
    SSD1306_UpdateScreen();
}
```

Solución extrema ☐ No compartir recursos

Las dos tareas que antes imprimían directamente en la pantalla ahora producen datos y los encolan en el fifo

```
void StartDefaultTask(void *argument)
{
    for (;;) {
        HAL_GPIO_TogglePin(GPIOC, GPIO_PIN_13);

        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13)) {
            //ImprimirLinea("LED = OFF", 1);
            mensaje_t dato={"LED = OFF",1};
        } else {
            //ImprimirLinea("LED = ON", 1);
            mensaje_t dato={"LED = ON",1};
        }
        osMessageQueuePut (miFIFOHandle, &dato, NULL, osWaitForever);
        osDelay(500);
    }
}
```

```
void entrySensor1(void *argument)
{
    int sensor = 0;
    char val[20] = "sensor1 = ";

    for (;;) {
        /* convierte el valor del sensor a ascii y lo concatena al
           final de "sensor2 = " */
        itoa(sensor, val + 10, 10);

        //ImprimirLinea(val, 2);
        mensaje_t dato={"",2};
        strcpy(dato.cad,val);
        osMessageQueuePut (miFIFOHandle, &dato, NULL, osWaitForever);

        sensor++;
        osDelay(100);
    }
}
```

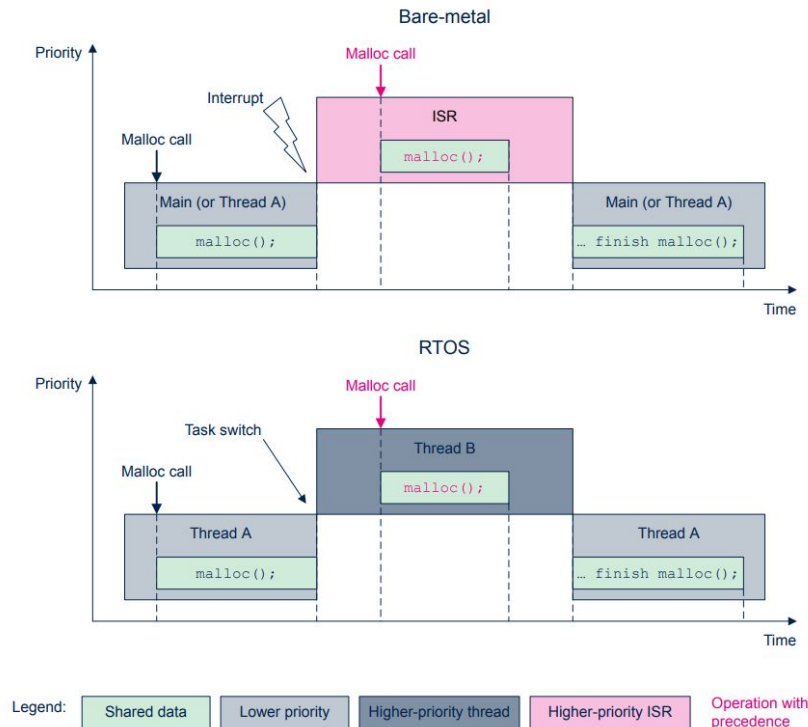
Uso de Librerías en sistemas concurrentes (funciones inseguras)

Al usar de librerías perdemos noción de lo que hace su código interno, pero siempre deberíamos asumir que no es seguro usarla libremente desde distintas tareas sin comprometer los datos o el correcto funcionamiento (no es **thread-safe**).

Cualquier función que haga uso de una variable global o estática (que no esté almacenado en el stack de la tarea), o un recurso de HW se considerará **no-reentrante**.

Conviene buscar librerías de terceros probadas en entornos multitareas, o escribir *wrappers* de las funciones que las conviertan en versiones seguras.

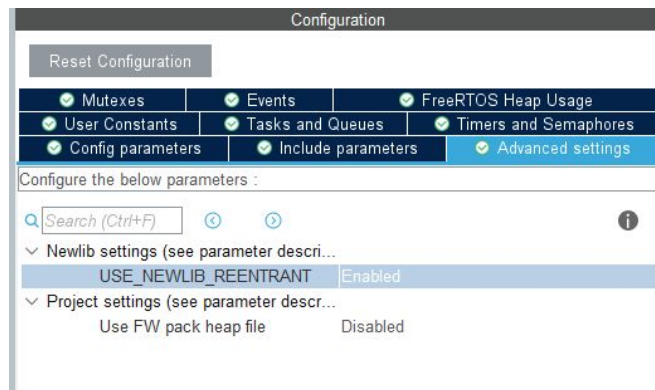
Uso de Librerías en sistemas concurrentes (funciones inseguras)



Uso de Librerías en sistemas concurrentes (funciones inseguras)

Si se van a utilizar las librerías estándar de C desde muchas tareas, hay que habilitar la opción `USE_NEWLIB_REENTRANT`

Solo funciona para concurrencia entre tareas, no con ISRs.



Uso de Librerías en sistemas concurrentes (funciones inseguras)

Para el uso seguro de la librería de C, incluyendo las ISRs, el CubeMX ofrece distintas estrategias

Figure 3. Thread-safe settings

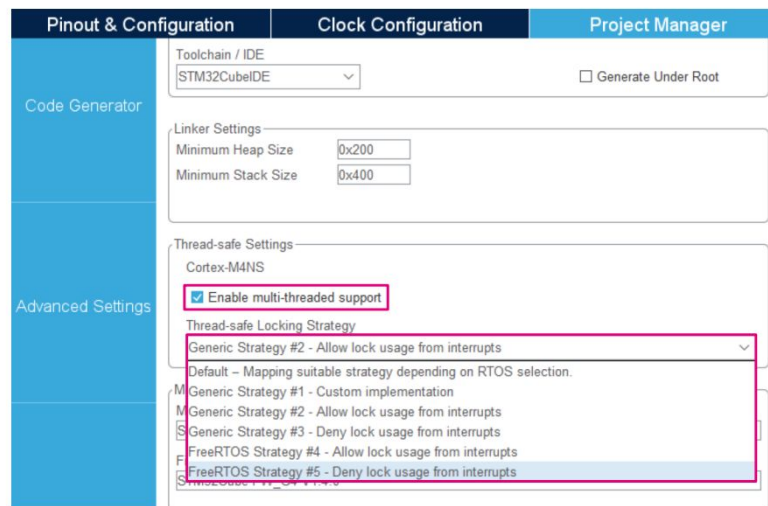
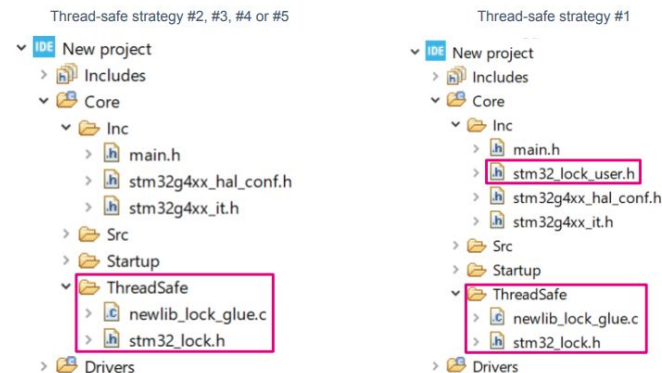


Figure 7. STM32CubeIDE single-core project configuration



Uso de Librerías en sistemas concurrentes (funciones inseguras)

La HAL, por otro lado, intenta implementar la exclusión mutua pero no lo hace correctamente, ya que no hay un acceso atómico a la variable de bloqueo.

stm32_hal_def.h

```
typedef enum
{
    HAL_UNLOCKED = 0x00U,
    HAL_LOCKED   = 0x01U
} HAL_LockTypeDef;

#if (USE_RTOS == 1U)
/* Reserved for future use */
#error "USE_RTOS should be 0 in the current HAL release"
#else
#define __HAL_LOCK(__HANDLE__) \
do{ \
    if((__HANDLE__)->Lock == HAL_LOCKED) \
    { \
        return HAL_BUSY; \
    } \
    else \
    { \
        (__HANDLE__)->Lock = HAL_LOCKED; \
    } \
}while (0U)

#define __HAL_UNLOCK(__HANDLE__) \
do{ \
    (__HANDLE__)->Lock = HAL_UNLOCKED; \
}while (0U)

#endif /* USE_RTOS */
```

```
HAL_StatusTypeDef HAL_I2C_Master_Transmit
(I2C_HandleTypeDef *hi2c, uint16_t DevAddress,
uint8_t *pData, uint16_t Size, uint32_t Timeout)
{
    if (hi2c->State == HAL_I2C_STATE_READY)
    {
        /* Preparativos de la comunicación*/

        /* Process Locked */
        __HAL_LOCK(hi2c);
        /****
        Comunicación insegura
        *****/
        __HAL_UNLOCK(hi2c);

        return HAL_OK;
    }
    else
    {
        return HAL_BUSY;
    }
}
```

stm32_hal_i2c.c

Práctica

1. Replicar el ejemplo de las diapositivas 8-9 y verificar los problemas de concurrencia
2. En proyectos distintos resuelva el problema de concurrencia utilizando:
 - a. Suspensión de interrupciones
 - b. Suspensión del scheduler
 - c. Mutex
 - d. Una única tarea que acceda al recurso compartido y un FIFO
3. Instale una nueva tarea en cada uno de los proyectos anteriores que imprima una tercera línea en la pantalla, intercalando entre su nombre y su apellido cada 200 ms. Verifique que se cumplan los tiempos propuestos para cada tarea.